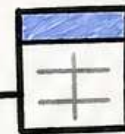
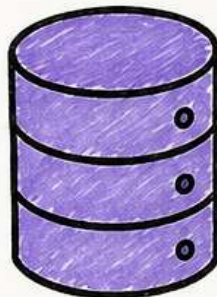
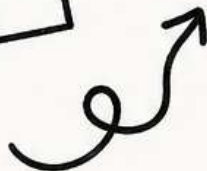
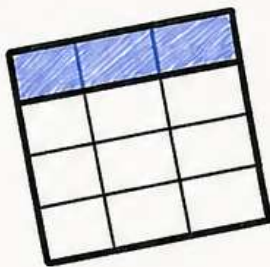




SQL NOTES



SELECT *
FROM
TABLE;



INDEX

SQL NOTES

S.No.	Topic	Page No.
1.	Introduction to SQL	1
2.	Database & RDBMS	2
3.	SQL Data Types	3
4.	CREATE DATABASE	4
5.	CREATE TABLE	5
6.	ALTER TABLE	6
7.	Constraints	7
8.	INSERT Statement	8
9.	UPDATE Statement	9
10.	DELETE Statement	10
11.	SELECT Statement	11
12.	WHERE Clause	12
13.	SQL Operators	13
14.	ORDER BY Clause	14
15.	DISTINCT Keyword	15
16.	LIMIT Clause	16
17.	Aggregate Functions	17
18.	GROUP BY Clause	18
19.	HAVING Clause	19
20.	Primary Key	20
21.	Foreign Key	21
22.	Other Keys	22
23.	Relationships	23
24.	INNER JOIN	24
25.	LEFT JOIN	25
26.	RIGHT JOIN	26
27.	FULL JOIN	27
28.	SELF JOIN	28
29.	Subqueries	29
30.	UNION & UNION ALL	30
31.	Views	31
32.	Indexes	32
33.	Transactions	33
34.	ACID Properties	34
35.	Normalization	35
36.	SQL vs NoSQL	36
37.	SQL Interview Questions	37
38.	SQL Cheat Sheet	38

LEARN
SQL

SELECT
*
FROM
TABLE;

INTRODUCTION TO SQL

- SQL (Structured Query Language) is a standard language used to communicate with databases.
- It is used to store, retrieve, update, delete, and manage data in a relational database.
- SQL is ANSI (American National Standards Institute) standard language.
- It is declarative in nature (you specify what you want, not how to do it).
- SQL is used in many systems like MySQL, PostgreSQL, Oracle, SQL Server, SQLite, etc.

What Can SQL Do?

- ✓ Create & modify databases
- ✓ Create & modify tables
- ✓ Insert, update & delete data
- ✓ Retrieve data using queries
- ✓ Control access to data

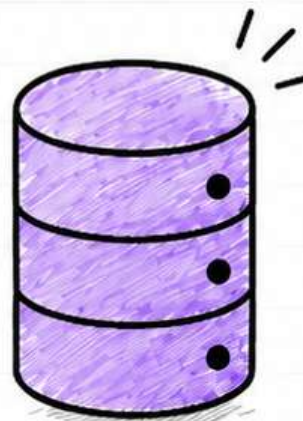


2

DATABASE & RDBMS

DATABASE

- A database is an **organized collection of related data**.
- It is stored and accessed electronically.
- It helps in easy storage, retrieval and management of data.



Example:

Library DB
Student DB
Company DB
etc.

RDBMS

- RDBMS stands for **Relational Database Management System**.
- It is a software used to create, manage and manipulate relational databases.
- Data is stored in **tables** (rows and columns).
- It maintains **relationships** between tables.

Table : Students

ID	Name	Age	City
1	Aman	20	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune

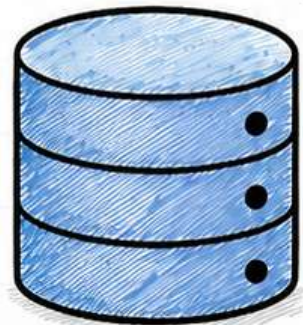
Examples of RDBMS

- MySQL
- PostgreSQL
- Oracle
- Microsoft SQL Server
- SQLite

3

SQL DATA TYPES

- Data types define the **kind of data** that can be stored in a column.
- They help in **efficient storage** and valid operations.
- SQL provides **different types** of data types to handle different kinds of values.



Data Type	Description	Example
INT / INTEGER	Stores whole numbers (positive, negative or zero).	10, -25, 0
DECIMAL(p, s)	Stores fixed-point numbers with precision (p) and scale (s).	12.50, -3.14
FLOAT / REAL	Stores decimal numbers with floating precision.	3.14159, -0.001
CHAR(n)	Stores fixed length string (n = length).	'A', 'SQL'
VARCHAR(n)	Stores variable length string (max length = n).	'Hello', 'Data'
DATE	Stores date in format YYYY-MM-DD.	2024-05-21
TIME	Stores time in format HH:MM:SS.	14:30:00
DATETIME	Stores date and time.	2024-05-21 14:30:00
BOOLEAN	Stores boolean values TRUE or FALSE.	TRUE, FALSE

4

CREATE DATABASE

WHAT IS CREATE DATABASE?

- The **CREATE DATABASE** statement is used to create a new database in SQL.



SYNTAX

```
CREATE DATABASE database_name;
```

EXAMPLE

```
CREATE DATABASE CollegeDB;
```

POINTS TO REMEMBER

- Database name must be **unique**.
- Use **meaningful names** for databases.
- After creation, you can use the database to **create tables** and store data.
- Different DBMS may have **different restrictions** on names.

Example Output

```
Query OK,  
1 row affected  
(0.00 sec)
```



5

CREATE TABLE

WHAT IS CREATE TABLE?

- The **CREATE TABLE** statement is used to create a new table in an existing database.
- It defines the columns and their structure in the table.

SYNTAX

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

EXAMPLE

```
CREATE TABLE Students (
    ID INT,
    Name VARCHAR(50),
    Age INT
);
```



KEY POINTS

- Table name should be **unique** within a database.
- Column names should be **meaningful** and follow naming rules.
- Every column must have a **datatype**.
- You can add **constraints** (like PRIMARY KEY, NOT NULL, etc.) later or while creating the table.
- After creation, the table is **empty** (no rows).

ANATOMY OF A TABLE

Table Name → **Students**

ID	Name	Age
1	John	20
2	Riya	21
3	Karan	22

Columns
(Fields)

Rows
(Records)

EXAMPLE OUTPUT

Query OK, 0 rows affected
(0.02 sec)

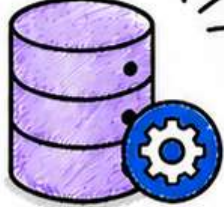


6

ALTER TABLE

WHAT IS ALTER TABLE?

- The **ALTER TABLE** statement is used to modify the structure of an existing table.
- It allows you to add, modify, drop or rename columns.



GENERAL SYNTAX

```
ALTER TABLE table_name
operation ... ;
```

EXAMPLE TABLE

Table Name : **Students**

ID	Name	Age
1	Aman	20
2	Riya	21
3	Karan	22

We will apply different **ALTER** operations on this table.

KEY POINTS

- ALTER TABLE** changes the **structure**, not the data.
- Existing data remains **safe**.
- Some operations may differ slightly based on the **DBMS**.
- Always **check** the table structure before making changes.

TYPES OF ALTER OPERATIONS

1. ADD COLUMN

Adds a new column in the table.

SYNTAX

```
ALTER TABLE table_name
ADD column_name datatype;
```

EXAMPLE

```
ALTER TABLE Students
ADD Email VARCHAR(100);
```

2. MODIFY COLUMN

Changes the data type or size of an existing column.

SYNTAX

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

EXAMPLE

```
ALTER TABLE Students
MODIFY Age BIGINT;
```

3. DROP COLUMN

Removes an existing column from the table.

SYNTAX

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

EXAMPLE

```
ALTER TABLE Students
DROP COLUMN Email;
```

4. RENAME COLUMN

Changes the name of an existing column.

SYNTAX

```
ALTER TABLE table_name
RENAME COLUMN old_name TO new_name;
```

EXAMPLE

```
ALTER TABLE Students
RENAME COLUMN Name TO FullName;
```


CONSTRAINTS

WHAT ARE CONSTRAINTS?

- Constraints are rules applied to columns in a table.
- They enforce data accuracy, integrity and reliability in the database.

WHY USE CONSTRAINTS?

- ✓ Ensure valid and correct data
- ✓ Prevent invalid data entry
- ✓ Maintain data integrity
- ✓ Improve database reliability

TYPES OF CONSTRAINTS

1. NOT NULL

Ensures that a column cannot have NULL value.

SYNTAX

column_name datatype NOT NULL

EXAMPLE

```
CREATE TABLE Students (
  Name VARCHAR(50) NOT NULL
);
```

Name	
John	✓
NULL	✗
Riya	✓

2. UNIQUE

Ensures that all values in a column are different.

SYNTAX

column_name datatype UNIQUE

EXAMPLE

```
CREATE TABLE Students (
  Email VARCHAR(100) UNIQUE
);
```

Email	
abc@gmail.com	✓
xyz@gmail.com	✓
abc@gmail.com	✗

3. PRIMARY KEY

Uniquely identifies each record in a table. It also implies NOT NULL and UNIQUE.

SYNTAX

column_name datatype PRIMARY KEY

EXAMPLE

```
CREATE TABLE Students (
  ID INT PRIMARY KEY,
  Name VARCHAR(50)
);
```



ID	
1	✓
2	✓
1	✗

4. FOREIGN KEY

Creates a link between two tables. It enforces referential integrity.

SYNTAX

column_name datatype
FOREIGN KEY (column_name)
REFERENCES table_name(column_name)

EXAMPLE

```
CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  StudentID INT,
  FOREIGN KEY (StudentID)
  REFERENCES Students(ID)
);
```

Students
ID
1
2
3

Orders

StudentID
1
2
3

StudentID in Orders must exist in Students.

5. DEFAULT

Sets a default value for a column when no value is provided.

SYNTAX

column_name datatype DEFAULT value

EXAMPLE

```
CREATE TABLE Students (
  Status VARCHAR(20) DEFAULT 'Active'
);
```

Status	
Active	✓
Active	✓
Active	✓

Default value applied.

6. CHECK

Ensures that all values in a column satisfy a condition.

SYNTAX

column_name datatype CHECK (condition)

EXAMPLE

```
CREATE TABLE Students (
  Age INT CHECK (Age >= 18)
);
```

Age	
20	✓
17	✗
18	✓

KEY POINTS

- ★ Constraints are applied at the time of table creation.
- ★ They can also be added later using ALTER TABLE.
- ★ They help in maintaining clean, valid and reliable data.
- ★ PRIMARY KEY and FOREIGN KEY are the most important constraints.

INSERT STATEMENT

WHAT IS INSERT?

- The **INSERT** statement is used to add new records (rows) into a table.

EXAMPLE TABLE

Table Name : **Students**

ID	Name	Age	City
1	Aman	20	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune

OUTPUT (After Example 2)

ID	Name	Age	City
1	Aman	20	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore
5	Rohit	24	Chennai
6	Priya	19	Kolkata

New Rows

EXAMPLE OUTPUT

Query OK, 1 row affected
Rows matched: 1 Changed: 1 Warnings: 0

SYNTAX

```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

EXAMPLES

1. Insert a single row

```
INSERT INTO Students (ID, Name, Age, City)
VALUES (4, 'Neha', 23, 'Bangalore');
```

2. Insert multiple rows

```
INSERT INTO Students (ID, Name, Age, City)
VALUES (5, 'Rohit', 24, 'Chennai'),
(6, 'Priya', 19, 'Kolkata');
```

3. Insert data without specifying column names

```
INSERT INTO Students
VALUES (7, 'Sahil', 20, 'Hyderabad');
```

NOTES

- ✓ The number of values must match the number of columns.
- ✓ The data type of values must match the column data type.
- ✓ If a column is NOT NULL and has no DEFAULT value, you must provide a value.
- ✓ You can insert one or more rows at a time.

UPDATE STATEMENT

WHAT IS UPDATE?

- The **UPDATE** statement is used to modify existing records in a table.

SYNTAX

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

EXAMPLE TABLE (Before Update)

Table Name : **Students**

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

OUTPUT (After Example 1)

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

Only
ID = 1
is
updated

EXAMPLES

1. Update a single column

```
UPDATE Students
SET Age = 21
WHERE ID = 1;
```

After
Update

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

2. Update multiple columns

```
UPDATE Students
SET City = 'Hyderabad', Age = 22
WHERE ID = 2;
```

After
Update

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	22	Hyderabad
3	Karan	22	Pune
4	Neha	23	Bangalore

3. Update all rows (Use with caution!)

```
UPDATE Students
SET City = 'India';
-- No WHERE clause
-- All rows will be updated
```

After
Update

ID	Name	Age	City
1	Aman	21	India
2	Riya	22	India
3	Karan	22	India
4	Neha	23	India

IMPORTANT NOTES

- WHERE** clause is important to update specific rows.
- Without **WHERE**, **all rows** will be updated.
- You can update one or more columns at a time.
- Data type of the new value must match the column data type.
- Always **double-check** conditions before running the query.

EXAMPLE OUTPUT

Query OK, 1 row affected
Rows matched: 1 Changed: 1 Warnings: 0

DELETE STATEMENT

WHAT IS DELETE?

- The **DELETE** statement is used to remove existing records (rows) from a table.

SYNTAX

```
DELETE FROM table_name
WHERE condition;
```

EXAMPLE TABLE (Before Delete)

Table Name : **Students**

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

OUTPUT (After Example 1)

Row with ID = 1 is deleted

ID	Name	Age	City
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

EXAMPLES

1. Delete a single row

```
DELETE FROM Students
WHERE ID = 1;
```

After Delete

ID	Name	Age	City
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

2. Delete rows based on condition

```
DELETE FROM Students
WHERE Age > 21;
```

After Delete

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai

3. Delete all rows from the table

```
DELETE FROM Students;
```

After Delete

ID	Name	Age	City
(No rows left)			

4. Delete all rows with a condition

```
DELETE FROM Students
WHERE City = 'Mumbai';
```

After Delete

ID	Name	Age	City
1	Aman	21	Delhi
3	Karan	22	Pune
4	Neha	23	Bangalore

IMPORTANT NOTES

- DELETE removes the **entire row** from the table.
- Without **WHERE** clause, **all rows** will be deleted.
- Use **WHERE** carefully to avoid losing important data.
- Deleted data **cannot be recovered** easily.
- Use **DELETE** with caution!

EXAMPLE OUTPUT

Query OK, 1 row affected
Rows matched: 1 Changed: 1 Warnings: 0

SELECT STATEMENT

WHAT IS SELECT?

- The **SELECT** statement is used to retrieve data from one or more tables in a database.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition; -- (optional)
```

EXAMPLE TABLE

Table Name : **Students**

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

COMMON CLAUSES

- SELECT** : Specifies the columns to display.
- FROM** : Specifies the table to retrieve data from.
- WHERE** : Filters records based on a condition.
- ORDER BY** : Sorts the result set.

IMPORTANT NOTES

- Use ***** to select all columns.
- Use meaningful column names instead of selecting unnecessary columns.
- WHERE** clause is optional.
- You can use multiple conditions using **AND**, **OR**, **NOT**.
- ORDER BY** can be **ASC** (default) or **DESC**.

EXAMPLES

1. Select all columns

```
SELECT *
FROM Students;
```

Result

ID	Name	Age	City
1	Aman	21	Delhi
2	Riya	21	Mumbai
3	Karan	22	Pune
4	Neha	23	Bangalore

2. Select specific columns

```
SELECT Name, Age
FROM Students;
```

Result

Name	Age
Aman	21
Riya	21
Karan	22
Neha	23

3. Select with WHERE condition

```
SELECT *
FROM Students
WHERE Age > 21;
```

Result

ID	Name	Age	City
3	Karan	22	Pune
4	Neha	23	Bangalore

4. Select specific columns with condition

```
SELECT Name, City
FROM Students
WHERE City = 'Mumbai';
```

Result

Name	City
Riya	Mumbai

5. Select with ORDER BY

```
SELECT Name, Age
FROM Students
ORDER BY Age ASC;
```

Result
(Ascending Order)

Name	Age
Aman	21
Riya	21
Karan	22
Neha	23

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Name, Age FROM Students WHERE Age > 21;
+-----+-----+
| Name | Age |
+-----+-----+
| Karan | 22  |
| Neha  | 23  |
+-----+-----+
2 rows in set (0.00 sec)
```


WHERE CLAUSE

WHAT IS WHERE?

- The **WHERE** clause is used to filter records and retrieve only those rows that satisfy a specified condition.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Only rows that meet the condition in **WHERE** clause will be returned.

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
4	Neha	22	Sales	40000
5	Rohit	35	HR	70000

COMPARISON OPERATORS

Operator	Meaning	Example
=	Equal to	Age = 25
<> or !=	Not equal to	Age <> 30
>	Greater than	Salary > 50000
<	Less than	Age < 30
>=	Greater than or equal to	Age >= 25
<=	Less than or equal to	Age <= 30

EXAMPLES (SINGLE CONDITION)

1. Employees older than 25

```
SELECT *
FROM Employees
WHERE Age > 25;
```

Result

ID	Name	Age	Dept	Salary
2	Riya	30	HR	60000
3	Karan	28	IT	55000
5	Rohit	35	HR	70000

2. Employees in IT department

```
SELECT *
FROM Employees
WHERE Dept = 'IT';
```

Result

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
3	Karan	28	IT	55000

3. Employees with salary >= 60000

```
SELECT *
FROM Employees
WHERE Salary >= 60000;
```

Result

ID	Name	Age	Dept	Salary
2	Riya	30	HR	60000
5	Rohit	35	HR	70000

4. Employees not in Sales department

```
SELECT *
FROM Employees
WHERE Dept <> 'Sales';
```

Result

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
5	Rohit	35	HR	70000

MULTIPLE CONDITIONS

Using AND

Returns rows that satisfy all the conditions.

```
SELECT *
FROM Employees
WHERE Age > 25
AND Salary > 50000;
```

Result

ID	Name	Age	Dept	Salary
2	Riya	30	HR	60000
5	Rohit	35	HR	70000

Using OR

Returns rows that satisfy any one of the conditions.

```
SELECT *
FROM Employees
WHERE Dept = 'IT'
OR Salary > 60000;
```

Result

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
3	Karan	28	IT	55000
5	Rohit	35	HR	70000

IMPORTANT NOTES

- WHERE** clause is optional, but very important for filtering data.
- Without **WHERE** clause, all rows will be returned.
- You can use comparison operators to compare values.
- You can combine multiple conditions using **AND**, **OR**, **NOT**.
- Use parentheses () to group conditions for better control.
- Text values should be in single quotes (' ').

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Name, Dept, Salary FROM Employees
WHERE Age > 25 AND Salary > 50000;
```

```
+-----+-----+-----+
| Name | Dept | Salary |
+-----+-----+-----+
| Riya | HR   | 60000  |
| Rohit | HR   | 70000  |
+-----+-----+-----+
```

2 rows in set (0.00 sec)

SQL OPERATORS

WHAT ARE SQL OPERATORS?

- SQL Operators are used to perform operations on values and variables in a SQL statement.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition using operator;
```

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Age	Salary	Dept
1	Aman	25	50000	IT
2	Riya	30	60000	HR
3	Karan	28	55000	IT
4	Neha	22	40000	Sales
5	Rohit	35	70000	HR

TYPES OF SQL OPERATORS

- Arithmetic Operators
- Comparison Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Special Operators

1. COMPARISON OPERATORS

Operator	Meaning	Example	Result
=	Equal to	Age = 25	TRUE/FALSE
<> / !=	Not equal to	Age <> 30	TRUE/FALSE
>	Greater than	Age > 25	TRUE/FALSE
<	Less than	Age < 30	TRUE/FALSE
>=	Greater than or equal to	Age >= 25	TRUE/FALSE
<=	Less than or equal to	Age <= 30	TRUE/FALSE

2. ARITHMETIC OPERATORS

Operator	Meaning	Example	Result
+	Addition	Salary + 5000	55000
-	Subtraction	Salary - 5000	45000
*	Multiplication	Salary * 2	100000
/	Division	Salary / 2	25000
%	Modulus (Remainder)	Salary % 2	0 or 1

4. BITWISE OPERATORS

Operator	Meaning	Example
&	Bitwise AND	A & B
	Bitwise OR	A B
^	Bitwise XOR	A ^ B
~	Bitwise NOT	~A
<<	Left Shift	A << 2
>>	Right Shift	A >> 2

Bitwise operators work on binary values (0 and 1). They are rarely used in most daily SQL queries.

6. SPECIAL OPERATORS

Operator	Meaning	Example
IN	Matches any value in a list	Dept IN ('IT', 'HR')
BETWEEN	Matches a range of values	Age BETWEEN 20 AND 30
LIKE	Pattern matching	Name LIKE 'A%'
IS NULL	Checks for NULL	Dept IS NULL
IS NOT NULL	Checks for NOT NULL	Dept IS NOT NULL
EXISTS	Checks existence of subquery result	EXISTS (SELECT * FROM T)
ALL	Compares with all values	Salary > ALL (SELECT Salary FROM Emp)
ANY / SOME	Compares with any value	Salary > ANY (SELECT Salary FROM Emp)

3. LOGICAL OPERATORS

Operator	Meaning	Example	Result
AND	Returns TRUE if both conditions are TRUE	Age > 25 AND Salary > 50000	TRUE/FALSE
OR	Returns TRUE if at least one condition is TRUE	Age > 25 OR Salary > 50000	TRUE/FALSE
NOT	Reverses the result	NOT (Age > 25)	TRUE/FALSE

5. ASSIGNMENT OPERATORS

Operator	Meaning	Example
=	Assign	@x = 10
+=	Add and Assign	@x += 5 (Same as @x = @x + 5)
-=	Subtract and Assign	@x -= 5 (Same as @x = @x - 5)
*=	Multiply and Assign	@x *= 2 (Same as @x = @x * 2)
/=	Divide and Assign	@x /= 2 (Same as @x = @x / 2)
%=	Modulus and Assign	@x %= 2 (Same as @x = @x % 2)

IMPORTANT NOTES

- Choose the right operator based on the requirement.
- Using operators in **WHERE** clause helps filter data effectively.
- Logical operators can be combined using parentheses for better control.
- Understanding operators is key to writing powerful SQL queries!

ORDER BY CLAUSE

WHAT IS ORDER BY?

- The **ORDER BY** clause is used to sort the result set in ascending (ASC) or descending (DESC) order based on one or more columns.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition; -- (optional)
ORDER BY column1 [ASC | DESC], column2 [ASC | DESC], ... ;
```

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
4	Neha	22	Sales	40000
5	Rohit	35	HR	70000

SORT ORDER

ASC (Ascending)
Sorts the data in smallest to largest (A to Z, 0 to 9).

DESC (Descending)
Sorts the data in largest to smallest (Z to A, 9 to 0).

ORDER BY MULTIPLE COLUMNS

You can sort by more than one column. If values in the first column are same, the second column is used for sorting.

```
SELECT *
FROM Employees
ORDER BY Dept ASC, Salary DESC;
```

Result

ID	Name	Age	Dept	Salary
3	Karan	28	IT	55000
1	Aman	25	IT	50000
5	Rohit	35	HR	70000
2	Riya	30	HR	60000
4	Neha	22	Sales	40000

IMPORTANT NOTES

- Default order is **ASC** if not specified.
- NULL** values are sorted first in **ASC** and last in **DESC** (varies by DBMS).
- Use **ORDER BY** to make output readable and meaningful.

EXAMPLES

1. Order by Age (Ascending)

```
SELECT *
FROM Employees
ORDER BY Age ASC;
```

2. Order by Age (Descending)

```
SELECT *
FROM Employees
ORDER BY Age DESC;
```

3. Order by Salary (Descending)

```
SELECT *
FROM Employees
ORDER BY Salary DESC;
```

4. Order by Dept, then Age (Ascending)

```
SELECT *
FROM Employees
ORDER BY Dept ASC, Age ASC;
```

5. Order by Name (Descending)

```
SELECT *
FROM Employees
ORDER BY Name DESC;
```

RESULT

ID	Name	Age	Dept	Salary
4	Neha	22	Sales	40000
1	Aman	25	IT	50000
3	Karan	28	IT	55000
2	Riya	30	HR	60000
5	Rohit	35	HR	70000

ID	Name	Age	Dept	Salary
5	Rohit	35	HR	70000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
1	Aman	25	IT	50000
4	Neha	22	Sales	40000

ID	Name	Age	Dept	Salary
5	Rohit	35	HR	70000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
1	Aman	25	IT	50000
4	Neha	22	Sales	40000

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
3	Karan	28	IT	55000
2	Riya	30	HR	60000
5	Rohit	35	HR	70000
4	Neha	22	Sales	40000

ID	Name	Age	Dept	Salary
5	Rohit	35	HR	70000
4	Neha	22	Sales	40000
3	Karan	28	IT	55000
2	Riya	30	HR	60000
1	Aman	25	IT	50000

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Name, Age, Dept, Salary FROM Employees ORDER BY Age ASC;
+-----+-----+-----+-----+
| Name | Age | Dept | Salary |
+-----+-----+-----+-----+
| Neha | 22 | Sales | 40000 |
| Aman | 25 | IT    | 50000 |
| Karan | 28 | IT    | 55000 |
| Riya | 30 | HR    | 60000 |
| Rohit | 35 | HR    | 70000 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```


DISTINCT KEYWORD

WHAT IS DISTINCT?

- The **DISTINCT** keyword is used to return only **unique** (different) values from a column or combination of columns.

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Dept	City	Salary
1	Aman	IT	Delhi	50000
2	Riya	HR	Mumbai	60000
3	Karan	IT	Delhi	55000
4	Neha	Sales	Pune	40000
5	Rohit	HR	Mumbai	60000
6	Riya	HR	Mumbai	60000
7	Aman	IT	Delhi	50000



IMPORTANT NOTES

- DISTINCT** works on one or more columns.
- NULL** values are considered as a **single** unique value.
- DISTINCT** is useful to remove duplicates and get a clean result set.
- DISTINCT** may reduce performance on large tables. Use it only when needed.

SYNTAX

```
SELECT DISTINCT column1, column2, ...
FROM table_name
WHERE condition; -- (optional)
```

DISTINCT

removes duplicate rows from the result set.

EXAMPLES

1. Get unique departments

```
SELECT DISTINCT Dept
FROM Employees;
```

Result

Dept
IT
HR
Sales

2. Get unique cities

```
SELECT DISTINCT City
FROM Employees;
```

Result

City
Delhi
Mumbai
Pune

3. Get unique salaries

```
SELECT DISTINCT Salary
FROM Employees;
```

Result

Salary
50000
60000
55000
40000

4. Get unique Dept and City combinations

```
SELECT DISTINCT Dept, City
FROM Employees;
```

Result

Dept	City
IT	Delhi
HR	Mumbai
Sales	Pune

5. Get unique Dept with condition

```
SELECT DISTINCT Dept
FROM Employees
WHERE Salary > 50000;
```

Result

Dept
HR
IT

MULTIPLE COLUMNS

When using **DISTINCT** with multiple columns, the **combination** of values must be unique to appear in the result.



DISTINCT vs ALL

Keyword	Meaning
DISTINCT	Returns only unique values (duplicates removed)
ALL	Returns all values including duplicates (default)

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT DISTINCT City FROM Employees;
+-----+
| City |
+-----+
| Delhi |
| Mumbai |
| Pune |
+-----+
3 rows in set (0.00 sec)
```



LIMIT CLAUSE

WHAT IS LIMIT?

- The **LIMIT** clause is used to restrict the number of rows returned by a query.

SYNTAX

```
SELECT column1, column2, ...
FROM table_name
WHERE condition; -- (optional)
ORDER BY column_name ASC | DESC; -- (optional)
LIMIT number_of_rows;
```

LIMIT is used to specify the maximum number of rows to return.

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
2	Riya	30	HR	60000
3	Karan	28	IT	55000
4	Neha	22	Sales	40000
5	Rohit	35	HR	70000
6	Pooja	27	IT	52000
7	Arjun	24	Sales	45000

Total rows in **Employees** table = 7

LIMIT with OFFSET (Optional)

OFFSET is used with LIMIT to skip a certain number of rows before starting to return rows.

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column_name;
LIMIT number_of_rows
OFFSET start_from_row;
```

Example

```
SELECT *
FROM Employees
ORDER BY ID ASC
LIMIT 3 OFFSET 2;
```

Result

ID	Name	Age	Dept	Salary
3	Karan	28	IT	55000
4	Neha	22	Sales	40000
5	Rohit	35	HR	70000

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT * FROM Employees ORDER BY Salary DESC LIMIT 3;
+----+-----+-----+-----+-----+
| ID | Name  | Age  | Dept | Salary |
+----+-----+-----+-----+-----+
| 5  | Rohit | 35   | HR   | 70000  |
| 2  | Riya  | 30   | HR   | 60000  |
| 3  | Karan | 28   | IT   | 55000  |
+----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

EXAMPLES

1. Get first 3 rows

```
SELECT *
FROM Employees
LIMIT 3;
```

Result

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
2	Riya	30	HR	60000
3	Karan	28	IT	55000

2. Get first 2 rows after sorting by Salary (Highest first)

```
SELECT *
FROM Employees
ORDER BY Salary DESC
LIMIT 2;
```

Result

ID	Name	Age	Dept	Salary
5	Rohit	35	HR	70000
2	Riya	30	HR	60000

3. Get first 3 rows of IT department

```
SELECT *
FROM Employees
WHERE Dept = 'IT'
LIMIT 3;
```

Result

ID	Name	Age	Dept	Salary
1	Aman	25	IT	50000
3	Karan	28	IT	55000
6	Pooja	27	IT	52000

4. Get first 2 rows of Sales department sorted by Age

```
SELECT *
FROM Employees
WHERE Dept = 'Sales'
ORDER BY Age ASC
LIMIT 2;
```

Result

ID	Name	Age	Dept	Salary
7	Arjun	24	Sales	45000
4	Neha	22	Sales	40000

IMPORTANT NOTES

- LIMIT** is very useful for pagination (showing data in pages).
- Always use **ORDER BY** with **LIMIT** to get consistent results.
- Without **ORDER BY**, the order of rows is not guaranteed.
- LIMIT** works in all major databases like **MySQL**, **PostgreSQL**, **SQLite**, etc.

AGGREGATE FUNCTIONS

WHAT ARE AGGREGATE FUNCTIONS?

- Aggregate functions perform a calculation on a set of values and return a **single** result.

SYNTAX

```
SELECT aggregate_function(column_name)
FROM table_name
WHERE condition; -- (optional)
GROUP BY column_name; -- (optional)
```

Aggregate functions ignore **NULL** values except **COUNT(*)**.

AGGREGATE FUNCTIONS

Function	Description	Example
COUNT()	Returns the number of rows	COUNT(*)
COUNT(column)	Returns the number of non-NULL values	COUNT(Salary)
SUM()	Returns the total sum of values	SUM(Salary)
AVG()	Returns the average of values	AVG(Age)
MIN()	Returns the smallest value	MIN(Salary)
MAX()	Returns the largest value	MAX(Age)

EXAMPLE TABLE

Table Name : **Employees**

ID	Name	Dept	Age	Salary
1	Aman	IT	25	50000
2	Riya	HR	30	60000
3	Karan	IT	28	55000
4	Neha	Sales	22	40000
5	Rohit	HR	35	70000
6	Pooja	IT	NULL	52000
7	Arjun	Sales	24	NULL

EXAMPLES

1. COUNT(*) - Count all rows

```
SELECT COUNT(*)
FROM Employees;
```

Result →

COUNT(*)
7

2. COUNT(Salary) - Count non-NULL salaries

```
SELECT COUNT(Salary)
FROM Employees;
```

Result →

COUNT(Salary)
6

3. SUM(Salary) - Total of salaries

```
SELECT SUM(Salary)
FROM Employees;
```

Result →

SUM(Salary)
327000

4. AVG(Age) - Average age

```
SELECT AVG(Age)
FROM Employees;
```

Result →

AVG(Age)
26.40

5. MIN(Salary) - Minimum salary

```
SELECT MIN(Salary)
FROM Employees;
```

Result →

MIN(Salary)
40000

6. MAX(Age) - Maximum age

```
SELECT MAX(Age)
FROM Employees;
```

Result →

MAX(Age)
35

USING GROUP BY

GROUP BY is used with aggregate functions to get grouped results.

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE condition; -- (optional)
GROUP BY column_name;
```

EXAMPLE

```
SELECT Dept, COUNT(*) AS Total_Employees,
AVG(Salary) AS Avg_Salary
FROM Employees
GROUP BY Dept;
```

Result →

Dept	Total_Employees	Avg_Salary
HR	2	65000.00
IT	3	52333.33
Sales	2	46000.00

IMPORTANT NOTES

- Aggregate functions always return a single value (except when used with GROUP BY).
- NULL values are ignored in COUNT(column), SUM(), AVG(), MIN(), MAX().
- COUNT(*) counts all rows including those with NULLs.
- Use GROUP BY to get aggregate results for each group.

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Dept, COUNT(*), SUM(Salary) FROM Employees GROUP BY Dept;
+----+-----+-----+
| Dept | COUNT(*) | SUM(Salary) |
+----+-----+-----+
| HR   | 2       | 130000      |
| IT   | 3       | 157000      |
| Sales | 2       | 92000       |
+----+-----+-----+
3 rows in set (0.00 sec)
```


GROUP BY CLAUSE

WHAT IS GROUP BY?

- The **GROUP BY** clause is used to group rows that have the same values into summary rows, like "find the total sales per department".

EXAMPLE TABLE

Table Name : **Sales**

ID	Name	Dept	Product	Amount
1	Aman	IT	Laptop	50000
2	Riya	HR	Chair	8000
3	Karan	IT	Mouse	2000
4	Neha	Sales	Laptop	45000
5	Rohit	HR	Chair	6000
6	Pooja	IT	Keyboard	1500
7	Arjun	Sales	Laptop	30000
8	Simran	HR	Mouse	1200
9	Vivek	IT	Laptop	55000

IMPORTANT NOTES

- ✓ **GROUP BY** groups rows with the same values into summary rows.
- ✓ It is usually used with aggregate functions like **COUNT()**, **SUM()**, **AVG()**, **MIN()**, **MAX()**.
- ✓ Column(s) in **SELECT** must be either in **GROUP BY** or used in an aggregate function.
- ✓ Use **HAVING** to filter grouped results.
- ✓ **WHERE** filters rows before grouping, **HAVING** filters groups after grouping.

HAVING vs WHERE

Feature	WHERE	HAVING
Purpose	Filters rows before grouping	Filters groups after grouping
Used with	Individual rows	Grouped results
Can use	Column names Only	Aggregate functions

SYNTAX

```
SELECT column1, aggregate_function(column2), ...
FROM table_name
WHERE condition; -- (optional)
GROUP BY column1, column2, ...
HAVING aggregate_condition; -- (optional)
ORDER BY column1; -- (optional)
```

GROUP BY works with aggregate functions to summarize data.

EXAMPLES

1. Total sales (Amount) by Department

```
SELECT Dept, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Dept;
```

Result

Dept	Total_Sales
HR	15200
IT	124500
Sales	75000

2. Total sales by Department and Product

```
SELECT Dept, Product, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Dept, Product;
```

Result

Dept	Product	Total_Sales
HR	Chair	14000
HR	Mouse	1200
IT	Keyboard	1500
IT	Laptop	105000
IT	Mouse	2000
Sales	Laptop	75000

3. Number of orders (rows) per Department

```
SELECT Dept, COUNT(*) AS Total_Orders
FROM Sales
GROUP BY Dept;
```

Result

Dept	Total_Orders
HR	3
IT	5
Sales	2

4. Average sales amount by Department

```
SELECT Dept, AVG(Amount) AS Avg_Amount
FROM Sales
GROUP BY Dept;
```

Result

Dept	Avg_Amount
HR	5066.67
IT	24900.00
Sales	37500.00

5. Departments with total sales more than 70000 (Using HAVING)

```
SELECT Dept, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Dept
HAVING SUM(Amount) > 70000;
```

Result

Dept	Total_Sales
IT	124500
Sales	75000

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Dept, SUM(Amount) AS Total_Sales FROM Sales GROUP BY Dept;
+----+-----+
| Dept | Total_Sales |
+----+-----+
| HR   | 15200      |
| IT   | 124500     |
| Sales | 75000      |
+----+-----+
3 rows in set (0.00 sec)
```


HAVING CLAUSE

WHAT IS HAVING?

- The **HAVING** clause is used to filter groups after aggregation. It is similar to **WHERE** but works with **GROUP BY**.

EXAMPLE TABLE

Table Name : Sales

ID	Name	Dept	Product	Amount
1	Aman	IT	Laptop	50000
2	Riya	HR	Chair	8000
3	Karan	IT	Mouse	2000
4	Neha	Sales	Laptop	45000
5	Rohit	HR	Chair	6000
6	Pooja	IT	Keyboard	1500
7	Arjun	Sales	Laptop	30000
8	Simran	HR	Mouse	1200
9	Vivek	IT	Laptop	55000

WHERE vs HAVING

Feature	WHERE	HAVING
Purpose	Filters individual rows before grouping	Filters groups after grouping
Used with	SELECT, without GROUP BY	SELECT with GROUP BY
Works on	Column values	Aggregate values (SUM, COUNT, AVG, etc.)
Execution Order	Before GROUP BY	After GROUP BY

IMPORTANT NOTES

- Use **WHERE** to filter rows, **HAVING** to filter groups.
- HAVING** can only be used with **GROUP BY**.
- Aggregate functions (SUM, COUNT, AVG, etc.) can be used in **HAVING**.
- You can use **WHERE** and **HAVING** together in the same query.

SYNTAX

```
SELECT column1, aggregate_function(column2), ...
FROM table_name
WHERE condition; -- (optional)
GROUP BY column1, column2, ...
HAVING aggregate_condition; -- (optional)
ORDER BY column1; -- (optional)
```

EXAMPLES

1. Departments with total sales greater than 50000

```
SELECT Dept, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Dept
HAVING SUM(Amount) > 50000;
```

Result

Dept	Total_Sales
IT	124500
Sales	75000

2. Departments with more than 2 orders

```
SELECT Dept, COUNT(*) AS Total_Orders
FROM Sales
GROUP BY Dept
HAVING COUNT(*) > 2;
```

Result

Dept	Total_Orders
IT	5
HR	3

3. Products with total sales at least 50000

```
SELECT Product, SUM(Amount) AS Total_Sales
FROM Sales
GROUP BY Product
HAVING SUM(Amount) >= 50000;
```

Result

Product	Total_Sales
Laptop	180000
Chair	14000

4. Departments with average sale amount greater than 10000

```
SELECT Dept, AVG(Amount) AS Avg_Amount
FROM Sales
GROUP BY Dept
HAVING AVG(Amount) > 10000;
```

Result

Dept	Avg_Amount
IT	24900.00
Sales	37500.00

EXAMPLE OUTPUT (MySQL)

```
mysql> SELECT Dept, SUM(Amount) AS Total_Sales
-> FROM Sales
-> GROUP BY Dept
-> HAVING SUM(Amount) > 50000;
+-----+-----+
| Dept | Total_Sales |
+-----+-----+
| IT   | 124500      |
| Sales | 75000       |
+-----+-----+
2 rows in set (0.00 sec)
```

HAVING is used to filter groups created by **GROUP BY** based on aggregate conditions.

PRIMARY KEY

WHAT IS PRIMARY KEY?

- A Primary Key **uniquely** identifies each row in a table.
- No **duplicate** values allowed.
- Cannot contain **NULL** values.

SYNTAX

```
CREATE TABLE Students (
  ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);
```

ID is the Primary Key (unique identifier)

EXAMPLE TABLE

Table Name : **Students**

Primary Key →

ID	Name	Age
1	Aman	20
2	Riya	21
3	Karan	22

RULES OF PRIMARY KEY

- ✓ Unique values only
- ✓ No **NULL** values
- ✓ One **Primary Key** per table
- ✓ Can contain **one or multiple** columns

CREATE TABLE with PRIMARY KEY

```
CREATE TABLE Students (
  ID INT PRIMARY KEY,
  Name VARCHAR(50),
  Age INT
);
```

ALTER TABLE - ADD PRIMARY KEY

```
ALTER TABLE Students
ADD PRIMARY KEY (ID);
```

COMPOSITE PRIMARY KEY

```
CREATE TABLE Enrollments (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID)
);
```

Combination of two or more columns as Primary Key

IMPORTANT NOTE

Primary Key = Unique Identity of each record

FOREIGN KEY

WHAT IS FOREIGN KEY?

- A **Foreign Key** is a column (or group of columns) in one table that refers to the **Primary Key** of another table.
- It helps to maintain **referential integrity** between tables.

SYNTAX

```
CREATE TABLE ChildTable (
  Column1 DataType,
  Column2 DataType,
  ForeignKeyColumn DataType,
  FOREIGN KEY (ForeignKeyColumn)
  REFERENCES ParentTable(PrimaryKeyColumn)
);
```

Child table refers to Parent table's Primary Key

EXAMPLE TABLES

Parent Table: **Departments**

DeptID (PK)	DeptName
1	IT
2	HR
3	Sales

Employees.DeptID is a **Foreign Key** that refers to Departments.DeptID (Primary Key)

Child Table: **Employees**

EmpID (PK)	EmpName	DeptID (FK)
101	Aman	1
102	Riya	2
103	Karan	1
104	Neha	3

CREATE TABLE with FOREIGN KEY

```
CREATE TABLE Departments (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);
CREATE TABLE Employees (
  EmpID INT PRIMARY KEY,
  EmpName VARCHAR(50),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

RULES OF FOREIGN KEY

- ✓ Must reference an existing **Primary Key**.
- ✓ Values in **Foreign Key** must exist in referenced table.
- ✓ Cannot insert a value that doesn't exist in **Parent** table.
- ✓ If **Primary Key** is deleted, related rows in **Child table** may be restricted.

ALTER TABLE - ADD FOREIGN KEY

```
ALTER TABLE Employees
ADD CONSTRAINT fk_emp_dept
FOREIGN KEY (DeptID)
REFERENCES Departments(DeptID);
```

IMPORTANT NOTES

- Foreign Key enforces **relationship** between tables.
- Helps to keep data **consistent** and accurate.
- A table can have **multiple** Foreign Keys.
- A table can be the **Parent** of many **Child** tables.

EXAMPLE OUTPUT

Departments		Employees		
DeptID	DeptName	EmpID	EmpName	DeptID
1	IT	101	Aman	1
2	HR	102	Riya	2
3	Sales	103	Karan	1
		104	Neha	3

DeptID in **Employees** references **DeptID** (Primary Key) in **Departments**.

OTHER KEYS

WHAT ARE OTHER KEYS?

- Besides **Primary Key** and **Foreign Key**, there are other types of keys used to maintain **data integrity** and **relationships** in a database.

1. UNIQUE KEY

- Ensures all values in a column (or group of columns) are **unique**.
- Allows **NULL** values (only one **NULL**).

SYNTAX

```
CREATE TABLE Users (
  Email VARCHAR(100) UNIQUE
);
```



3. ALTERNATE KEY

- Candidate keys **excluding the Primary Key**.
- Used when there are **multiple** Candidate Keys.

EXAMPLE

If RollNo is Primary Key, then Email (candidate key) becomes an Alternate Key.



5. COMPOSITE (COMPOUND) KEY

- A Primary Key made of two or more columns.
- Used when no single column can uniquely identify a row.

SYNTAX

```
CREATE TABLE Enrollments (
  StudentID INT,
  CourseID INT,
  PRIMARY KEY (StudentID, CourseID)
);
```



IMPORTANT NOTES

- Primary Key is the best Candidate Key.
- Alternate Keys are backup options.
- Foreign Key maintains relationship.
- Choose the right key for data accuracy and performance.

TYPES OF KEYS

2. CANDIDATE KEY

- A column (or set of columns) that can uniquely identify rows in a table.
- A table can have **multiple** Candidate Keys, but only one **Primary Key**.

EXAMPLE

In a Students table, both RollNo and Email can uniquely identify a student. So, RollNo and Email are Candidate Keys.



4. SUPER KEY

- Any set of columns that can uniquely identify rows in a table.
- May include **extra columns**.

EXAMPLE

In **Students** (RollNo, Email, Name), (RollNo), (Email), (RollNo, Email), (RollNo, Name) are Super Keys.

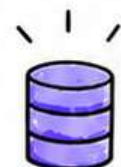


6. SURROGATE KEY

- An artificial key with no business meaning.
- Usually an auto-incremented ID.

EXAMPLE

```
ID INT PRIMARY KEY AUTO_INCREMENT
(used just to uniquely identify rows)
```



QUICK RECAP

- Primary Key** → Unique + Not NULL + One per table
- Foreign Key** → Refers to Primary Key of another table
- Unique Key** → Unique values (one NULL allowed)
- Candidate Key** → Can uniquely identify rows
- Alternate Key** → Candidate Key (not Primary)
- Super Key** → Any combination that uniquely identifies
- Composite Key** → Key using 2 or more columns
- Surrogate Key** → Artificial / system generated key

RELATIONSHIPS

SQL

WHAT IS RELATIONSHIP?

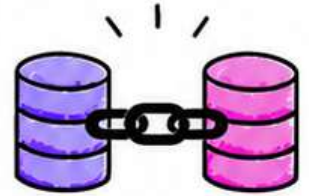
- A Relationship is an **association** between two tables.
- It is implemented using **Primary Key** and **Foreign Key**.

KEY IDEA

Primary Key in one table

Foreign Key in another table

creates a **Relationship**



TYPES OF RELATIONSHIPS

1. ONE-TO-ONE (1:1)

- One record in Table A is related to **One** record in Table B.
- Example: Person ↔ Passport



EXAMPLE

Each Person has one Passport and each Passport belongs to one Person.



2. ONE-TO-MANY (1:N)

- One record in Table A is related to **Many** records in Table B.
- Example: Department → Employees



EXAMPLE

One Department can have many Employees, but each Employee belongs to one Department.



3. MANY-TO-MANY (M:N)

- Many records in Table A are related to **Many** records in Table B.
- Example: Students ↔ Courses



EXAMPLE

Many Students can enroll in many Courses and many Courses can have many Students.



HOW M:N IS IMPLEMENTED?

- Many-to-Many relationship is implemented using a **Junction (Bridge) Table**.
- It contains **Foreign Keys** of both tables and usually a **Composite Primary Key**.



EXAMPLE: STUDENTS ↔ COURSES

Students		Enrollments		Courses	
StudentID (PK)	Name	StudentID (FK)	CourseID (FK)	CourseID (PK)	CourseName
1	Aman	1	101	101	SQL
2	Riya	1	102	102	Python
3	Karan	2	101	103	DSA
		3	102		
		3	103		

(PK: StudentID, CourseID)

IMPORTANT NOTES

- Relationships maintain data integrity.
- They reduce data redundancy.
- Use Foreign Keys to link related tables.
- Choose the correct relationship type based on real-world requirements.

QUICK RECAP

- 1:1** → One record in A to One record in B.
- 1:N** → One record in A to Many records in B.
- M:N** → Many records in A to Many records in B (use Junction Table).

INNER JOIN

WHAT IS INNER JOIN?

- INNER JOIN returns only the **matching** rows from both tables.
- If there is no match, the row is **not included** in the result.

SYNTAX

```
SELECT column_list
FROM table1
INNER JOIN table2
ON table1.column = table2.column;
```

EXAMPLE TABLES

Employees Table

EmpID (PK)	EmpName	DeptID (FK)
1	Aman	10
2	Riya	20
3	Karan	10
4	Neha	30

EmpID is **Primary Key** in Employees table.
DeptID is **Foreign Key** referencing DeptID in Departments table.

Departments Table

DeptID (PK)	DeptName
10	IT
20	HR
30	Sales
40	Finance

INNER JOIN QUERY

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

EXPLANATION

- Matches Rows where Employees.DeptID is equal to Departments.DeptID.
- Returns only the **matching rows** from both tables.

RESULT (OUTPUT)

EmpID	EmpName	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Sales

DeptID = 40 (Finance) has no matching employee, so it is **not included**.

★ Only matching rows are returned in **INNER JOIN**.

IMPORTANT NOTES

- ✓ Inner Join returns only the intersecting (matching) rows.
- ✓ It is the most commonly used Join.
- ✓ Use table aliases to write shorter and clear queries.

QUICK RECAP

- ★ **INNER JOIN** = Match found (rows returned)
- ★ **No Match** = Row **excluded**
- ★ **Key Clause** = **ON**

LEFT JOIN

SQL

WHAT IS LEFT JOIN?

- LEFT JOIN returns all the rows from the left table and the matching rows from the right table.
- If there is no match, NULL values are returned for columns from the right table.

SYNTAX

```
SELECT column_list
FROM table1
LEFT JOIN table2
ON table1.column = table2.column;
```

EXAMPLE TABLES

Employees Table

EmpID (PK)	EmpName	DeptID (FK)
1	Aman	10
2	Riya	20
3	Karan	10
4	Neha	30

EmpID is Primary Key
in Employees table.
DeptID is Foreign Key
referencing DeptID in
Departments table.

Departments Table

DeptID (PK)	DeptName
10	IT
20	HR
30	Sales
40	Finance

LEFT JOIN QUERY

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EXPLANATION

- Returns all rows from the left table (Employees).
- Returns matching rows from the right table (Departments).
- If no match is found, NULL is returned for columns from the right table.

RESULT (OUTPUT)

EmpID	EmpName	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Sales
5	Rohit	NULL

- ★ If an employee has a DeptID that does not exist in Departments, the DeptName will be NULL.
- ★ Here, Rohit (EmpID 5) belongs to DeptID 50 which does not exist, so DeptName is NULL.

IMPORTANT NOTES

- LEFT JOIN always returns at least as many rows as the left table.
- It keeps all data from the left table safe.
- Use table aliases to write shorter queries.

QUICK RECAP

- ★ LEFT JOIN = All from Left + Matched from Right
- ★ No Match on Right = NULL
- ★ Key Clause = ON

RIGHT JOIN

SQL

WHAT IS RIGHT JOIN?

- RIGHT JOIN returns all the rows from the right table and the matching rows from the left table.
- If there is no match, NULL values are returned for columns from the left table.

SYNTAX

```
SELECT column_list
FROM table1
RIGHT JOIN table2
ON table1.column = table2.column;
```

EXAMPLE TABLES

Employees Table

EmpID (PK)	EmpName	DeptID (FK)
1	Aman	10
2	Riya	20
3	Karan	10
4	Neha	30

EmpID is Primary Key in Employees table.
DeptID is Foreign Key referencing DeptID in Departments table.

Departments Table

DeptID (PK)	DeptName
10	IT
20	HR
30	Sales
40	Finance

RIGHT JOIN QUERY

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
RIGHT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EXPLANATION

- Returns all rows from the right table (Departments).
- Returns matching rows from the left table (Employees).
- If no match is found, NULL is returned for columns from the left table.

RESULT (OUTPUT)

EmpID	EmpName	DeptName
1	Aman	IT
3	Karan	IT
2	Riya	HR
4	Neha	Sales
NULL	NULL	Finance

- ★ DeptID 40 (Finance) does not have any employee, so EmpID and EmpName are NULL.
- ★ All departments are included, even if there are no matching employees.

IMPORTANT NOTES

- RIGHT JOIN always returns at least as many rows as the right table.
- It keeps all data from the right table safe.
- Use table aliases to write cleaner queries.

QUICK RECAP

- ★ RIGHT JOIN = All from Right + Matched from Left
- ★ No Match on Left = NULL
- ★ Key Clause = ON

FULL JOIN

WHAT IS FULL JOIN?

- **FULL JOIN** returns all rows when there is a match in **either left table** or **right table**.
- If there is no match, **NULL** values are returned for the missing side.

SYNTAX

```
SELECT column_list
FROM table1
FULL JOIN table2
ON table1.column = table2.column;
```

EXAMPLE TABLES

Employees Table

EmpID (PK)	EmpName	DeptID (FK)
1	Aman	10
2	Riya	20
3	Karan	10
4	Neha	30
5	Rohit	50

EmpID is **Primary Key** in **Employees** table.
DeptID is **Foreign Key** referencing DeptID in **Departments** table.

Departments Table

DeptID (PK)	DeptName
10	IT
20	HR
30	Sales
40	Finance

FULL JOIN QUERY

```
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
FULL JOIN Departments D
ON E.DeptID = D.DeptID;
```

EXPLANATION

- Returns **all rows** from both tables.
- Matches rows where DeptID is same.
- Unmatched rows from either table are also included.
- **NULL** is returned for columns from the other table when there is no match.

RESULT (OUTPUT)

EmpID	EmpName	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Sales
5	Rohit	NULL
NULL	NULL	Finance

- ★ EmpID 5 (Rohit) has DeptID 50 which does not exist in Departments → DeptName is **NULL**.
- ★ DeptID 40 (Finance) does not have any employee → EmpID and EmpName are **NULL**.
- ★ So, **all** rows from both tables are returned.

IMPORTANT NOTES

- ✓ **FULL JOIN** = **LEFT JOIN** + **RIGHT JOIN**.
- ✓ Returns every row from both tables.
- ✓ Very useful when you don't want to miss any data from either table.

QUICK RECAP

- ★ **FULL JOIN** ⇒ All from Left + All from Right
- ★ Match → Normal rows returned
- ★ No Match → **NULLs** for missing side
- ★ Key Clause = **ON**

SELF JOIN

WHAT IS SELF JOIN?

- A table is joined with **itself** using **different aliases**.
- Useful for hierarchical or relational data within the same table.

SYNTAX

```
SELECT column_list
FROM table_name T1
JOIN table_name T2
ON T1.common_column = T2.common_column;
```

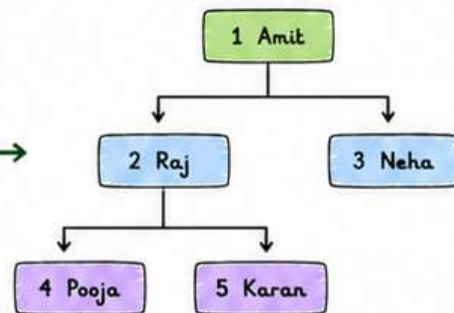
EXAMPLE TABLE

Employees table (Hierarchy Example)

EmpID (PK)	EmpName	ManagerID (FK)
1	Amit	NULL
2	Raj	1
3	Neha	1
4	Pooja	2
5	Karan	2

ManagerID is a Foreign Key that refers to EmpID in the same table. (Self Reference)

HIERARCHY VIEW



SELF JOIN QUERY

```
SELECT E.EmpName AS Employee,
       M.EmpName AS Manager
FROM Employees E
LEFT JOIN Employees M
ON E.ManagerID = M.EmpID
ORDER BY E.EmpID;
```

EXPLANATION

- The table **Employees** is joined with itself.
- **E** is alias for Employee (subordinate).
- **M** is alias for Manager.
- Matches ManagerID with EmpID.
- Shows each employee with their manager.

RESULT (OUTPUT)

Employee	Manager
Amit	NULL
Raj	Amit
Neha	Amit
Pooja	Raj
Karan	Raj

IMPORTANT NOTES

- ✓ Self Join is used for hierarchical data.
- ✓ Always use aliases to distinguish the same table.
- ✓ Can be combined with INNER, LEFT, RIGHT JOINS.
- ✓ Common use cases: Employee-Manager, Category Hierarchy, Organizational Structure.

QUICK RECAP

- ★ Self Join = Table with itself.
- ★ Use aliases to differentiate.
- ★ Useful for Hierarchical Relationships.
- ★ Key Column = ON clause matches within same table.

Self Join =
Table joins
itself using
different
aliases.

SUBQUERIES

WHAT IS A SUBQUERY?

- A subquery is a query nested inside another query.
- The inner query is executed first and its result is used by the outer query.
- Subqueries can be used in **SELECT**, **FROM**, **WHERE**, **HAVING** clauses.

SYNTAX

```
SELECT column_list
FROM table_name
WHERE column operator
( subquery );
```

EXAMPLE TABLE

Employees

EmpID (PK)	EmpName	DeptID	Salary
1	Aman	10	60000
2	Riya	20	50000
3	Karan	10	70000
4	Neha	30	45000
5	Rohit	20	55000
6	Pooja	30	65000

Find employees who earn more than the average salary.

SUBQUERY

```
SELECT AVG(Salary)
FROM Employees;
```

Returns a single value

SUBQUERY IN WHERE CLAUSE

```
SELECT EmpName, Salary
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

RESULT (OUTPUT)

EmpName	Salary
Raran	70000
Rohit	55000
Pooja	65000

TYPES OF SUBQUERIES

- **Single Row Subquery**
Returns zero or one row.
- **Multiple Row Subquery**
Returns one or more rows.
- **Multiple Column Subquery**
Returns one or more columns.
- **Correlated Subquery**
Depends on the outer query.

SUBQUERY IN FROM CLAUSE

```
SELECT DeptID, AvgSalary
FROM (
    SELECT DeptID, AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DeptID
) AS DeptAvg
WHERE AvgSalary > 50000;
```

RESULT (OUTPUT)

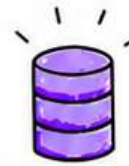
DeptID	AvgSalary
10	65000
20	52500

KEY POINTS

- ✓ Subqueries make complex queries easier to write and understand.
- ✓ Use parentheses to enclose subqueries.
- ✓ Subqueries can be nested.
- ✓ They improve readability and help break logic into steps.

QUICK RECAP

- ★ Subquery = Query inside another query.
- ★ Inner query runs first.
- ★ Use in **SELECT**, **FROM**, **WHERE**, **HAVING**.
- ★ Can return single value, multiple rows, or multiple columns.
- ★ Useful for filtering, comparisons, and calculations.



UNION and UNION ALL

WHAT ARE UNION AND UNION ALL?

- **UNION** and **UNION ALL** are used to combine the result sets of two or more **SELECT** queries.
- Both queries must have the same number of columns with compatible data types.
- Column names in the result are taken from the first **SELECT**.

SYNTAX

UNION (removes duplicates)

SELECT column1, column2, ... **FROM** table1

UNION

SELECT column1, column2, ... **FROM** table2;



UNION ALL (keeps duplicates)

SELECT column1, column2, ... **FROM** table1

UNION ALL

SELECT column1, column2, ... **FROM** table2;

EXAMPLE TABLES

Employees_Dept1

EmpID	EmpName	Dept
1	Aman	IT
2	Riya	IT
3	Karan	IT



Employees_Dept2

EmpID	EmpName	Dept
3	Karan	HR
4	Neha	HR
5	Rohit	HR

NOTE

- Both **SELECT** statements have the same number of columns (EmpID, EmpName, Dept) and compatible data types.



UNION (REMOVES DUPLICATES)

SELECT EmpID, EmpName, Dept **FROM** Employees_Dept1

UNION

SELECT EmpID, EmpName, Dept **FROM** Employees_Dept2;



RESULT (OUTPUT)

EmpID	EmpName	Dept
1	Aman	IT
2	Riya	IT
3	Karan	IT
4	Neha	HR
5	Rohit	HR

- ★ Duplicate rows are removed.

- ★ EmpID 3 (Karan) appears only once.

UNION ALL (KEEPS DUPLICATES)

SELECT EmpID, EmpName, Dept **FROM** Employees_Dept1

UNION ALL

SELECT EmpID, EmpName, Dept **FROM** Employees_Dept2;



RESULT (OUTPUT)

EmpID	EmpName	Dept
1	Aman	IT
2	Riya	IT
3	Karan	IT
3	Karan	HR
4	Neha	HR
5	Rohit	HR

- ★ All rows are returned, including duplicates.

- ★ EmpID 3 (Karan) appears twice. (once from each table)

KEY POINTS

- ✓ **UNION** removes duplicate rows.
- ✓ **UNION ALL** keeps all rows (including duplicates).
- ✓ Column names come from the first **SELECT**.
- ✓ Both **SELECT** statements must have the same number of columns with compatible data types.
- ✓ **ORDER BY** can be used at the end of the final query (not in the individual **SELECT**s).

USING ORDER BY WITH UNION

SELECT EmpName **FROM** Employees_Dept1

UNION

SELECT EmpName **FROM** Employees_Dept2

ORDER BY EmpName; -- Applied to the final result



QUICK RECAP

- ★ **UNION** = Combine + Remove Duplicates
- ★ **UNION ALL** = Combine + Keep Duplicates

- ★ Same number of columns
- ★ **ORDER BY** at the end (optional)



31 VIEWS



WHAT IS A VIEW?

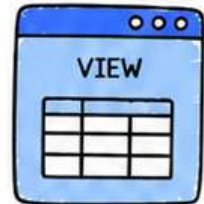
- A view is a **virtual table** based on the result of a **SELECT** query.
- It does not store data physically.
- It **simplifies complex queries** and enhances **security**.



SYNTAX

```
-- Create View
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;

-- Use View
SELECT * FROM view_name;
```



EXAMPLE TABLES

Employees Table

EmpID	EmpName	DeptID	Salary
1	Aman	10	60000
2	Riya	20	50000
3	Karan	10	70000
4	Neha	30	45000
5	Rohit	20	55000

Departments Table

DeptID	DeptName	Location
10	IT	Delhi
20	HR	Mumbai
30	Sales	Pune

EXAMPLE: CREATE VIEW

```
-- View to show employee names with
department names
CREATE VIEW EmpDeptView AS
SELECT E.EmpID, E.EmpName, D.DeptName
FROM Employees E
JOIN Departments D
ON E.DeptID = D.DeptID;
```



USING THE VIEW

```
-- Retrieve data from the view
SELECT * FROM EmpDeptView;

-- With additional filtering
SELECT EmpName, DeptName
FROM EmpDeptView
WHERE DeptName = 'IT';
```



RESULT (FROM EmpDeptView)

EmpID	EmpName	DeptName
1	Aman	IT
2	Riya	HR
3	Karan	IT
4	Neha	Sales
5	Rohit	HR

HOW IT WORKS

- The view runs the stored **SELECT** query each time it is used.
- It always reflects the latest data from base tables.
- Views do not increase storage usage for data.



BENEFITS OF VIEWS

- ✓ Simplify complex queries.
- ✓ Enhance security (restrict access to data).
- ✓ Provide data abstraction.
- ✓ Ensure consistency.
- ✓ Make queries easier to read and maintain.

UPDATABLE VIEWS

- Some views can be updated like tables.
- Typically a view is updatable if:
 - ★ It is based on a single table.
 - ★ It does not use **DISTINCT**, **GROUP BY**, **HAVING**, or aggregate functions.
 - ★ It does not use **UNION**.



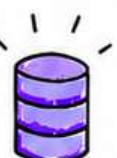
DROP A VIEW

```
-- Remove a view
DROP VIEW view_name;
```



QUICK RECAP

- ★ Create View = **CREATE VIEW view_name AS SELECT ...**
- ★ Use View = **SELECT * FROM view_name;**
- ★ Update View Data = If the view is updatable
- ★ Remove View = **DROP VIEW view_name;**
- ★ Views = **Virtual Tables**
- ★ Always reflect **current data** from base tables.





INDEXES



WHAT IS AN INDEX?

- An **Index** is a database object used to speed up data retrieval.
- It works like an **index** in a book, helping SQL find data quickly.
- Improves **SELECT** query performance.
- Stored separately from table data.



SYNTAX

-- Create Index

```
CREATE INDEX index_name
ON table_name(column_name);
```

-- Remove Index

```
DROP INDEX index_name
ON table_name;
```



EXAMPLE TABLE

Students

ID	Name	City
1	Aman	Delhi
2	Riya	Mumbai
3	Karan	Pune
4	Neha	Bangalore
5	Rohit	Delhi

Create Index on City
 CREATE INDEX idx_city
 ON Students(City);



HOW IT WORKS

Without Index ✗

```
SELECT * FROM Students
WHERE City = 'Delhi';
```

→ SQL checks every row one by one.



With Index ✓

```
SELECT * FROM Students
WHERE City = 'Delhi';
```

→ SQL directly jumps to matching records.



TYPES OF INDEXES

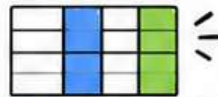
1. Single Column Index

```
CREATE INDEX idx_city
ON Students(City);
```



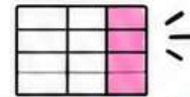
2. Composite Index

```
CREATE INDEX idx_name_city
ON Students(Name, City);
```



3. Unique Index

```
CREATE UNIQUE INDEX idx_email
ON Users(Email);
```



BENEFITS OF INDEXES

- ✓ Faster data retrieval
- ✓ Improves **SELECT** performance
- ✓ Efficient searching
- ✓ Better sorting and filtering



IMPORTANT NOTES

- Too many indexes can slow **INSERT**, **UPDATE**, and **DELETE** operations.
- Indexes consume extra storage.
- Use indexes on frequently searched columns.



QUICK RECAP

- ★ Index = Faster Search
- ★ Similar to Book Index
- ★ Best for **WHERE**, **ORDER BY**, **JOIN**
- ★ Improves Read Performance
- ★ Excessive Indexes may slow Writes



Find Data



Use Index



Fast Search!



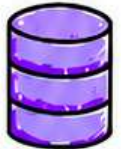
TRANSACTIONS

WHAT IS A TRANSACTION?

- A Transaction is a sequence of SQL operations performed as a single unit of work.
- Either all operations are completed successfully (**COMMIT**) or none of them are (**ROLLBACK**).
- Ensures data consistency and reliability.

ACID PROPERTIES

- ★ **A - Atomicity** : All or nothing.
- ★ **C - Consistency** : Brings DB from one valid state to another.
- ★ **I - Isolation** : Transactions are isolated from each other.
- ★ **D - Durability** : Once committed, changes are permanent.



BASIC SQL COMMANDS

BEGIN or START TRANSACTION; -- Starts a transaction
COMMIT; -- Saves all changes permanently
ROLLBACK; -- Undoes changes made in the transaction
SAVEPOINT name; -- Sets a savepoint
ROLLBACK TO name; -- Rolls back to a savepoint



EXAMPLE SCENARIO

Transfer ₹1000 from Account A to Account B

- 1 Deduct ₹1000 from Account A
- 2 Add ₹1000 to Account B
- 3 If both steps succeed → **COMMIT**
If any step fails → **ROLLBACK**



EXAMPLE: USING TRANSACTION

BEGIN TRANSACTION; -- Start transaction
UPDATE Accounts SET Balance = Balance - 1000 WHERE AccountNo = 'A'; -- Deduct from A
UPDATE Accounts SET Balance = Balance + 1000 WHERE AccountNo = 'B'; -- Add to B
COMMIT; -- Make changes permanent



EXAMPLE: ROLLBACK ON ERROR

BEGIN TRANSACTION;
UPDATE Accounts SET Balance = Balance - 1000 WHERE AccountNo = 'A';
 -- Suppose an error occurs here
UPDATE Accounts SET Balance = Balance + 1000 WHERE AccountNo = 'B';
ROLLBACK; -- All changes undone



SAVEPOINT EXAMPLE

BEGIN TRANSACTION;
UPDATE Accounts SET Balance = Balance - 1000 WHERE AccountNo = 'A';
SAVEPOINT after_deduct;
UPDATE Accounts SET Balance = Balance + 1000 WHERE AccountNo = 'B';
 -- If something goes wrong after this point
ROLLBACK TO after_deduct; -- Roll back to savepoint
COMMIT;



TYPES OF TRANSACTIONS

- Implicit Transaction
Automatically handled by the DBMS (e.g., single SQL statement).
- Explicit Transaction
Manually controlled using **BEGIN**, **COMMIT**, **ROLLBACK**.
- Nested Transaction (via Savepoints)
Transactions within transactions using savepoints.



BENEFITS

- ✓ Maintains data integrity.
- ✓ Prevents partial updates.
- ✓ Allows safe recovery in case of errors.
- ✓ Supports concurrent data access.



IMPORTANT NOTES

- Always **COMMIT** a transaction if all operations are successful.
- Use **ROLLBACK** to undo incorrect operations.
- Keep transactions short to avoid locking and performance issues.



QUICK RECAP

- ★ Transaction = Unit of Work
- ★ **ACID** ensures reliability
- ★ **COMMIT** = Save changes
- ★ **ROLLBACK** = Undo changes
- ★ Use transactions for safe and consistent operations





ACID PROPERTIES

WHAT IS ACID?

- ACID is a set of four properties that guarantee reliability of transactions in a database.
- These properties ensure that transactions are processed reliably and data remains consistent even in case of errors or failures.

THE FOUR ACID PROPERTIES

- ★ A - Atomicity
- ★ C - Consistency
- ★ I - Isolation
- ★ D - Durability



PROPERTY	MEANING	EXAMPLE	KEY POINT
A Atomicity 	All or nothing. Either all operations in a transaction are completed successfully or none of them are.	Example: Transfer ₹1000 from A to B • If both debit from A and credit to B succeed → COMMIT • If any step fails → ROLLBACK (no balance is changed)	No partial updates. The transaction is treated as a single unit.
C Consistency 	Brings the database from one valid state to another valid state. All rules, constraints and integrity are maintained.	Example: If Account balance cannot be negative, the transaction will not violate this rule. Only valid data is allowed.	Database remains consistent before and after the transaction.
I Isolation 	Transactions are isolated from each other. Concurrent transactions do not affect each other.	Example: Two users updating same account at the same time → They don't see each other's intermediate changes.	Each transaction behaves as if it is the only one running.
D Durability 	Once a transaction is committed, changes are permanent even if system crashes.	Example: After COMMIT, even if power goes off, the data remains saved.	Committed data is never lost.

BENEFITS OF ACID

- Ensures data reliability and integrity.
- Prevents data corruption.
- Helps in error recovery.
- Supports concurrent users safely.
- Builds trust in database systems.



WHERE IS ACID USED?

- Banking Systems
- E-commerce
- Online Booking Systems
- Any system that handles critical data

QUICK RECAP

- ★ A - Atomicity : All or nothing.
- ★ C - Consistency : Always valid
- ★ I - Isolation : Separate execution
- ★ D - Durability : Permanent changes



ACID properties work together to ensure **Safe, Reliable** and **Consistent** Transactions.



Better
Design,
Better Data!

35

NORMALIZATION

SQL

WHAT IS NORMALIZATION?

- Normalization is the process of organizing data in a database to reduce **redundancy** and improve **data integrity**.
- It divides large tables into smaller, well-structured tables and defines relationships between them.

WHY NORMALIZATION?

- ✓ Eliminates duplicate data
- ✓ Reduces data redundancy
- ✓ Improves data consistency
- ✓ Reduces update anomalies (Insert, Update, Delete)
- ✓ Saves storage space
- ✓ Makes database easy to maintain

NORMAL FORMS (UP TO 3NF)

1NF - FIRST NORMAL FORM

- Each column must have atomic (indivisible) values.
- No repeating groups or arrays.
- Each record must be unique.

Example (Not in 1NF)

ID	Name	Phones
1	Aman	9876, 8765
2	Riya	9988, 7766

After 1NF

ID	Name	Phone
1	Aman	9876
1	Aman	8765
2	Riya	9988
2	Riya	7766

2NF - SECOND NORMAL FORM

- Must be in 1NF.
- No partial dependency.
- All non-key attributes depend on the whole primary key.

Example (Not in 2NF)

OrderID	ProductID	ProductName	Quantity
101	P1	Pen	2
101	P2	Book	1

After 2NF

OrderID	ProductID	Quantity
101	P1	2
101	P2	1

ProductID	ProductName
P1	Pen
P2	Book

3NF - THIRD NORMAL FORM

- Must be in 2NF.
- No transitive dependency.
- Non-key attributes should depend only on the primary key.

Example (Not in 3NF)

EmpID	EmpName	DeptID	DeptName
1	Aman	10	IT
2	Riya	20	HR

After 3NF

EmpID	EmpName	DeptID
1	Aman	10
2	Riya	20

DeptID	DeptName
10	IT
20	HR

NORMALIZATION ANOMALIES

- ✗ **Insert Anomaly** : Cannot insert data without other data.
- ✗ **Update Anomaly** : Same data updated in multiple rows.
- ✗ **Delete Anomaly** : Deleting one row may cause loss of other data.

DECOMPOSITION RULES

- ✓ **Lossless Join** : Joining tables should not lose any data.
- ✓ **Dependency Preservation** : All dependencies must be preserved.
- ✓ **Smaller Tables** : Each table should have a clear purpose.

KEY TERMS

- **Super Key** : Set of attributes that uniquely identify a record.
- **Candidate Key** : Minimal Super Key.
- **Primary Key** : Candidate key chosen as the main key.
- **Foreign Key** : Key that links one table to another.

QUICK RECAP

- ★ Normalization organizes data.
- ★ It has 1NF, 2NF, 3NF (most commonly used).
- ★ Helps in reducing redundancy and anomalies.
- ★ Always follow decomposition rules.
- ★ Well-normalized tables = Better performance and data integrity.

Different Databases, Different Needs!

36

SQL vs NoSQL

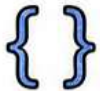
WHAT IS SQL?

- SQL stands for Structured Query Language.
- It is a traditional relational database.
- Data is stored in tables (rows & columns).
- Uses SQL for defining and manipulating data.



WHAT IS NoSQL?

- NoSQL stands for Not Only SQL.
- It is a non-relational or distributed database.
- Data can be stored in many formats: Key-Value, Document, Column, Graph.
- Designed for big data and high scalability.



SQL vs NoSQL COMPARISON

Feature	SQL (Relational DB)	NoSQL (Non-Relational DB)
1. Data Structure	Table based (Rows & Columns)	Various (Key-Value, Document, Column, Graph)
2. Schema	Fixed schema	Dynamic schema
3. Language	Uses SQL	Does not use SQL (Uses specific APIs/Queries)
4. Scalability	Vertical Scaling (Scale up)	Horizontal Scaling (Scale out)
5. Relationships	Uses joins (Relationships)	No joins (Denormalized data)
6. ACID Compliance	Full ACID compliant	Usually BASE (Basically Available, Soft state, Eventually consistent)
7. Best For	Complex queries, transactions	Big data, high performance, real-time apps
8. Examples	MySQL, Oracle, SQL Server, PostgreSQL	MongoDB, Cassandra, Redis, CouchDB
9. Data Consistency	High consistency	Eventual consistency
10. Use Cases	Banking, Finance, ERP, CRM	Social Media, IoT, Real-time analytics, Logs

SQL EXAMPLE (Relational Table)

Table: Students

ID	Name	City	Age
1	Aman	Delhi	20
2	Riya	Mumbai	21
3	Karan	Pune	22

- Structured data
- Fixed columns
- Easy to relate



NoSQL EXAMPLE (Document - MongoDB)

Collection: Students

```
{
  "id": 1,
  "name": "Aman",
  "city": "Delhi",
  "age": 20
}
```

```
{
  "id": 2,
  "name": "Riya",
  "city": "Mumbai",
  "age": 21
}
```

- Flexible data
- No fixed schema
- Easy to scale



ADVANTAGES

SQL

- ✓ Strong consistency
- ✓ ACID compliant
- ✓ Complex query support
- ✓ Mature & stable
- ✓ Data integrity

NoSQL

- ✓ Highly scalable
- ✓ Flexible schema
- ✓ High performance
- ✓ Handles big data
- ✓ Cost effective



WHEN TO USE?

Use SQL when:

- You need complex queries
- Data is structured
- Transactions are important
- Consistency is critical

Use NoSQL when:

- You need high scalability
- Data is unstructured or semi-structured
- High performance is key
- Handling big data / real-time data



QUICK RECAP

SQL = Structured, Reliable, Consistent
↓
Best for traditional applications

NoSQL = Flexible, Scalable, Fast
↓
Best for modern, big data applications



Choose the Right Tool for the Right Job!



Practice
Today,
Crack
Tomorrow!



37



SQL INTERVIEW QUESTIONS

BASIC QUESTIONS

- 1 What is SQL?
- 2 What is a database?
- 3 What is a table?
- 4 What is the difference between CHAR and VARCHAR?
- 5 What is the difference between DELETE, TRUNCATE and DROP?
- 6 What is a NULL value?
- 7 What is the primary key?
- 8 What is the foreign key?



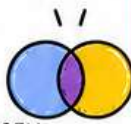
QUERY BASED QUESTIONS

- 1 Write a query to fetch all columns from a table.
- 2 Write a query to fetch only unique records.
- 3 Write a query to fetch top 5 records.
- 4 Write a query to find the second highest salary.
- 5 Write a query to count total number of rows.
- 6 Write a query to fetch records in a range.
- 7 Write a query to sort data in descending order.
- 8 Write a query to find duplicate records.



JOINS QUESTIONS

- 1 What is a JOIN?
- 2 What are the types of JOINS in SQL?
- 3 Write a query for INNER JOIN.
- 4 Write a query for LEFT JOIN.
- 5 Write a query for RIGHT JOIN.
- 6 Write a query for FULL OUTER JOIN.



FUNCTIONS QUESTIONS

- 1 What are aggregate functions? Give examples.
- 2 Write a query using COUNT(), SUM(), AVG(), MIN(), MAX().
- 3 What is the difference between HAVING and WHERE?
- 4 What is the use of GROUP BY?
- 5 What is the use of DISTINCT?
- 6 What is a subquery?



NORMALIZATION QUESTIONS

- 1 What is normalization?
- 2 What are the normal forms in DBMS?
- 3 What is 1NF?
- 4 What is the purpose of normalization?



ADVANCED QUESTIONS

- 1 What is a stored procedure?
- 2 What is a trigger?
- 3 What is the difference between UNION and UNION ALL?
- 4 What is indexing?
- 5 What is a transaction?



CONSTRAINTS QUESTIONS

- 1 What is a constraint?
- 2 What are the types of constraints?
- 3 What is the use of UNIQUE constraint?
- 4 What is the use of CHECK constraint?
- 5 What is the use of DEFAULT constraint?



SCENARIO / REAL LIFE QUESTIONS

- 1 Write a query to find the employees who earn more than the average salary.
- 2 Write a query to find Nth highest salary.
- 3 Write a query to find employees who joined in the last 6 months.
- 4 Write a query to find customers who have not placed any order.



TIPS TO CRACK SQL INTERVIEW

- ★ Practice queries daily.
- ★ Understand concepts, don't just memorize.
- ★ Solve problems on real datasets.
- ★ Focus on writing optimized queries.



- ★ Know basic to advanced concepts.
- ★ Practice on platforms like LeetCode, HackerRank, SQLZoo, StrataScratch.
- ★ Revise important topics regularly.



